

A Rapid Prototyping Approach for Smart Energy Systems based on Partial System Models

Georg Hackenberg, Maximilian Irlbeck and Vasileios Koutsoumpas
Chair IV: Software & Systems Engineering
Technische Universität München
Boltzmannstraße 3
85748 Garching, Germany
Email: {hackenbe,irlbeck,koutsoum}@in.tum.de

Denis Bytschkow
fortiss GmbH
Guerickestraße 25
80805 Munich, Germany
Email: bytschkow@fortiss.org

Abstract—A key success factor for building smart control software for today’s energy systems is a precise understanding of their behavioral requirements. However, current engineering approaches lack the possibility to validate these requirements in early phases of development. The validation is hampered by the difficulty estimating the impact of requirements on the overall system behavior before a complete system model is built. To overcome this situation, we propose a model-based rapid prototyping approach based on partial system models enabling early validation of behavioral requirements through tool-supported behavior estimation. For evaluating the approach we demonstrate our approach through a case study showing the interaction between customer and software engineer over three iterations. Finally, the approach is discussed with respect to modeling effort and required model resolution.

Keywords—*Rapid prototyping, smart grid.*

I. INTRODUCTION

Nowadays the paradigm of central energy production is shifting [1]. Decentralized and renewable, but fluctuating energy sources require a smart system operation. Thus, smart operation includes both ensuring critical properties (e.g. line and transformer capacities), while meeting conflicting objectives (e.g. emission reduction, microgrid autonomy, etc.). The specification of these requirements and their translation to the system level is the central goal of requirements engineering [2]. However, it is known that in many cases requirements are stated only in natural language baring the danger of ambiguity and misinterpretation. To overcome this situation, for complex systems rapid prototyping offers an iterative approach to requirements engineering, which allows one to reduce uncertainty, ambiguity, and inconsistency inherent in the engineering process [3]. Chua et al. [4] define a prototype as an approximation of a product (or system) or its components in some form for a definite purpose in its implementation. Laszewski et al. [5] were one of the first to identify the need for dedicated rapid prototyping support for the smart grid domain. Recently, researchers have been picking up on this idea contributing in several directions: For example, Vale et al. [6] study the effect of decentralized decision making on energy management. In contrast, Vaccaro et al. [7] provide an integrated framework for microgrid control including verification capabilities. Moreover, Faschang et al. [8] propose message bus technology for prototyping networked control systems. Finally, Elsheikh et al. [9] evaluate the effectiveness of Modelica for prototyping physical

energy system components. Overall, these approaches help the design of technical solutions, which is why they are suitable particularly for later phases of engineering projects. However, to the best of our knowledge at present time none of the rapid prototyping approaches for the smart grid domain focuses on the requirements validation phase. Requirements validation plays a key role when the impact of informal requirement statements (e.g. “certain devices need to be controlled smartly”) cannot be estimated: For example, customers might expect huge return-on-investments from smart control solutions, while in reality operational constraints and environmental conditions hinder the actual impact. Therefore, our research objective is to explore to what extent model-based rapid prototyping can be exploited for impact estimation driven requirements validation in early phases. The long-term goal is to establish an agile requirements engineering method that utilizes model-based, formal techniques to come up with valid specifications and high customer confidence quickly.

In the following we present the theoretic foundation before describing the model-based rapid prototyping approach. The approach itself consists of a model definition (using a generic, reusable model architecture) and a model analysis step embedded into an iterative workflow between customer and software engineer. The model architecture prescribes five types of models for the structuring of prototypes, namely context, constraint, objective, (partial) software and equivalence models. For model analysis, a behavior optimizer is provided to estimate the system behavior, which is based on stochastic optimization techniques. The behavior optimizer is responsible for resolving the variables of the partial system models. We evaluate the approach on a case study describing the interaction between customer and software engineer over three iterations. For each iteration the customer inputs, the prototype models and the analysis results are shown. Finally, the effort needed for modeling and the resulting model validity are discussed.

II. FOUNDATION

We base our approach on FOCUS [10], a model-based and strictly formal software and systems engineering method. The method builds on a mathematical theory, which describes systems in terms of their structure and behavior. The system structure is determined by a static hierarchy of components, each defining an interface through a set of input ports $I \in \mathbb{I}$ and output ports $O \in \mathbb{O}$. Ports $P \in \mathbb{I} \cup \mathbb{O}$ essentially are

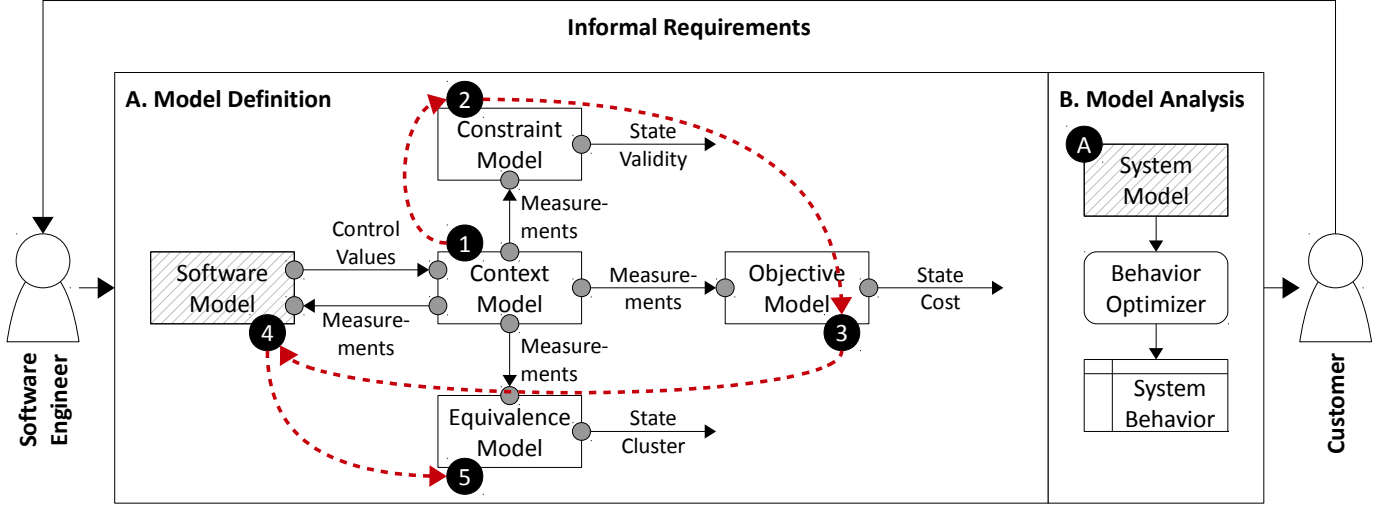


Fig. 1. Model-based rapid prototyping workflow including customer and software engineer communicating through informal requirements, model definition (i.e. context, constraint, objective, partial software and equivalence models) and model analysis (i.e. partial system model, behavior optimizer and system behavior).

message streams over the time domain $\mathbb{T}$ and the message domain $\mathbb{M}$, i.e. $P : \mathbb{T} \rightarrow \mathbb{M}$. Typically, for software systems the discrete time domain $\mathbb{T} = \mathbb{N}$ is used to reflect the discrete semantics of computer programs. Further, the ports of different components can be connected through channels to describe component interaction through message exchange. Finally, component behavior is determined by a mapping from the set of possible input streams over the input ports to a set of possible output streams over the output ports.

The application of the FOCUS method to the smart grid domain was already demonstrated in [11]. In particular, a fundamental model structure divided into context system (i.e. technical hardware and environment) and software system (i.e. control logic) was introduced. Further, a simplified, but scalable electric model has been proposed modeling energy production and consumption in a market-like fashion neglecting critical properties such as reactive power. Finally, the model used predefined component behavior of thermal devices reacting to changes in environment temperature only with more or less frequent air conditioning cycles. Consequently, in [12] the formal specification of optimization problems on top of FOCUS system models was shown using an annotation-based approach. In particular, the approach allows one to describe the objective function as well as the constraints of the optimization problem using regular output ports of components. Similarly, the variables of the optimization problem are defined as the behavior of the software system. Consequently, system models can be analyzed with respect to cost-effect system behavior. Based on this work we propose a rapid prototyping approach for smart energy systems comprising a generic, reusable model architecture and systematic engineering workflow.

III. RAPID PROTOTYPING APPROACH

Similar to existing rapid prototyping approaches, the goal of our approach is to develop prototypes from informal customer requirements quickly to get valuable feedback early in the development process. Compared to existing approaches, to decrease modeling effort we propose to use partial system models and respective analysis techniques.

Figure 1 illustrates the iterative workflow including customer, software engineer, informal requirements and model definition (based on a generic, reusable model architecture) as well as model analysis. The *customer* is responsible for expressing requirements for the system under development typically through a textual/informal requirements document. In the beginning, he defines an initial system vision, which is refined iteratively through information gained about prototypes provided by the software engineer. This iterative process ends as soon as the desired system properties are reached. The *software engineer*, on the other hand, is responsible for analyzing the informal requirements documents received from the customer, deriving respective prototypes and delivering back information about their operative behavior. As mentioned earlier, to achieve these goals in a systematic and repeatable manner the software engineer carries out two activities: (1) Model definition and (2) model analysis. Finally, the *informal requirements* typically state in natural language what is expected from the solution delivered by the software engineer. Ideally, the requirements are precise enough to enable a solid acceptance testing procedure. However, in our approach we particularly deal with requirements that are far from being formal enough to enable this kind of verification, which is rather typical in many industrial settings.

A. Model Definition

In a first step, the **context model** is defined so as to describe the environment of the software being developed. From a black box viewpoint, context models contain the mapping from control values to measurements. The actual modeling technique employed for describing this mapping depends on the system properties that need to be shown. We rely on discrete-time and hybrid-state models, which typically are sufficient in early phases as argued by Campetelli et al. [13].

In a second step, a **constraint model** is added to specify the hard limits of system operation, i.e. states that have to be avoided by all means. Therefore, constraint models specify a mapping from context measurements to a single validity output. Example constraints are maximum line capacities,

maximum consumption and production balance deviation at system level, minimum and maximum frequency or maximum voltage at certain measurement points. Constraints effectively filter the set of possible software behavior through statements about context state.

In a third step, an **objective model** is specified to capture the goals of system operation. The goal is expressed in terms of a cost or value output, which is derived from context measurement inputs. Examples are maximization of device service availability, minimization of CO₂ emissions, minimization of fossil resource usage or maximization of local net balance. Objectives effectively sort the set of filtered software behavior through a cost (i.e. minimization) or value (i.e. maximization) statement about context behavior.

In a fourth step, the **software model** is added to describe the prototype-specific behavior of the software. In general, the task of the software is to define a mapping from context measurements to control values. To decrease modelling effort, at this stage in particular non-deterministic or probabilistic models are supported allowing a range of control value outputs for each time step. These ranges effectively model the variables of the constrained optimization problem, which need to be resolved during model analysis.

In a fifth step, an **equivalence model** can be defined to enhance the optimizer results. The optimizer essentially searches the output space of the software model. Usually, this space is rather large, such that an exhaustive search is not feasible. Consequently, the optimizer allows us to cluster the resulting context states and follows clusters rather than individual states. Equivalence models enable the software engineer to influence how states are clustered through deriving an equivalence vector from context measurements.

B. Model Analysis

The *behavior optimizer* is responsible for finding particularly good behavior with respect to the constraints and the objective for a predefined time period. Consequently, techniques from the field of mathematical optimization have to be employed. Since in early phases we do not want to make assumptions about the optimization problem, we choose to use derivative-free stochastic optimization methods. Amongst others, these methods have already been shown to be effective for energy planning [14].

The *system behavior* finally is described in terms of two-dimensional tables containing the input and output port valuations for all components in the component hierarchy of the system model over the predefined time period. To reduce data complexity and increase data comprehensibility the tables serve as input for various visualization techniques, which are not described in this article.

IV. CASE STUDY

For evaluating the presented approach we developed a case study showing the interaction between customer and software engineer over three iterations. Note that the case study is intended to demonstrate a potential real-world scenario, however both customers and software engineers were members of the same group of researchers.

The system under development is a local net consisting of ten thermal devices as power consumers (e.g. air conditioning), one solar panel as well as one energy storage. The customer envisions to build a smart control system, which adjusts the energy consumption to the available production to reach a high degree of autonomy. To reach this goal, the hardware devices shall be extended with control logic that optimizes their collective behavior with respect to the common objective and respective constraints.

A. Iteration 1

The first iteration starts with the informal requirement to use the energy storage component as the only smart device in the system. A single controller can be realized with low development efforts and only little communication overhead. Further, a short description for each device type is available. The description contains a brief overview and the basic constraints of the devices. In other words it specifies the energy consumption of thermal devices and their minimum and maximum temperature limits, the maximum capacity of the energy storage, and the maximum production rate of the solar panel. In addition the customer argues that the power flow in the local energy net can be calculated as the sum of the energy consumption / production of individual devices.

1) *Model Definition*: First, the time resolution of the system model is determined. In the given case, discrete intervals of 15 minutes were selected for two reasons: (1) Energy prices typically are provided in equal resolution. (2) Context states could be described sufficiently accurately with this resolution.

The **context model** itself is composed of four different component types as shown in Figure 2: Local net, solar panel, energy storage and thermal device. The *local net* calculates for each time step an aggregate energy balance from the input loads (production or consumption) of the connected devices.

$$Balance(t) = \sum_{i=1}^n Load_i(t)$$

Production is modelled as positive load, consumption as negative load. Thus, the balance gives an estimate of how much energy flows *from* or *to* the local net.

The *solar panel* models the production as one instance of renewable energy sources. A typical solar panel operates at its maximal efficiency rate η_{max} .

$$Load_{Solar}(t) = \eta_{max} \cdot SolarEnergy(t)$$

Hereby, the solar energy is modelled as a Gaussian with the mean at 1:00 pm and a covariance of 7 h. According to the informal requirements the peak production is set to 3.0 kW.

Further, the *energy storage* adds variable energy production, consumption and buffer capabilities. The energy storage behavior depends on the charge decision input. Production can be observed in discharge mode, consumption can be observed in charge mode, and zero load can be observed in idle mode:

$$Load_{Storage}(t) = \begin{cases} 2.0 \text{ kW} & \text{if } Charge(t) = Discharge \\ -2.0 \text{ kW} & \text{if } Charge(t) = Charge \\ 0.0 \text{ kW} & \text{if } Charge(t) = Idle \end{cases}$$

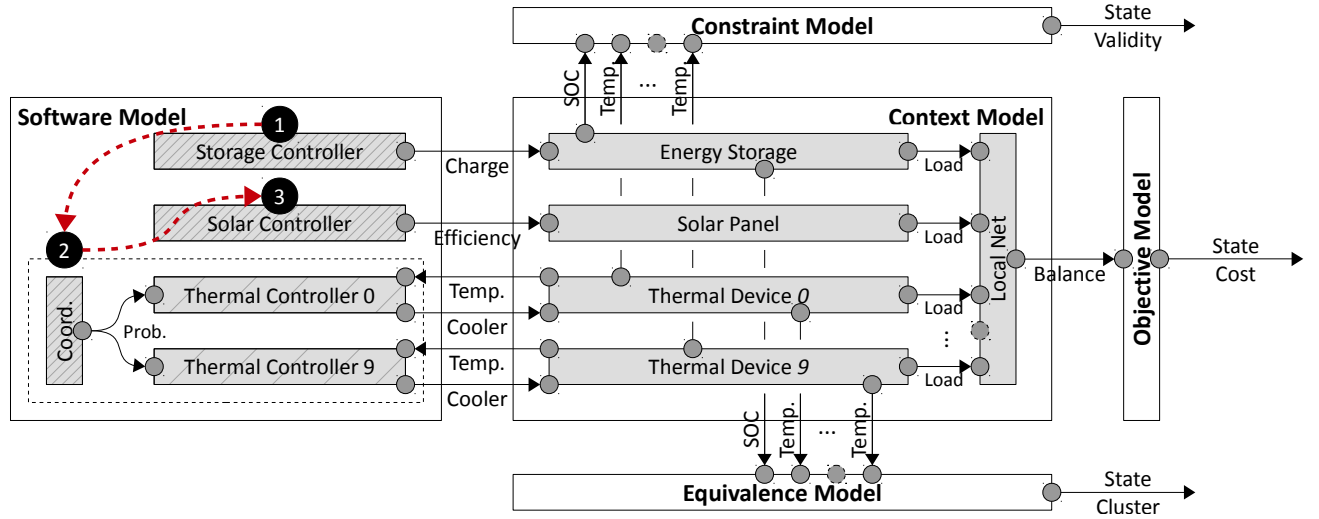


Fig. 2. Partial system models over three iterations from (1) smart storage over (2) smart storage + smart thermal devices to (3) smart storage + smart thermal devices + smart solar panel including deterministic (no hatching), non-deterministic (narrow hatching) and probabilistic (loose hatching) components.

The effects on the state of charge (SOC) are modeled under consideration of a variable charge efficiency $\eta(t)$ and a loss factor l . The variable charge efficiency $\eta(t)$ is defined as:

$$\eta(t) = \begin{cases} 1.0 & \text{if } Charge(t) = Discharge \\ 0.5 & \text{if } Charge(t) = Charge \end{cases}$$

In contrast, the constant loss factor l is set to 99%. Finally, both parameters are used to describe the temporal behavior of the state of charge:

$$SOC(t) = SOC(t-1) \cdot l - \eta(t) \cdot Load_{Storage}(t)$$

Finally, the state of charge is initialized with 50% of the energy storage's overall capacity (see constraint model).

At last, the *thermal devices* model the consumption as an instance of controllable loads. The thermal device consumption depends on the cooler decision:

$$Load_{Thermal}(t) = \begin{cases} 0W & \text{if } Cool(t) = Off \\ 200W & \text{if } Cool(t) = On \end{cases}$$

Depending on the decision the inner temperature increases or decreases by a constant ($\Delta T_+ = 0.5^\circ C$ and $\Delta T_- = 1.5^\circ C$):

$$Temp(t) = \begin{cases} Temp(t-1) + \Delta T_{Off} & \text{if } Cool(t) = Off \\ Temp(t-1) - \Delta T_{On} & \text{if } Cool(t) = On \end{cases}$$

Finally, the temperature of all thermal devices is initialized with $5^\circ C$. Note that without hindering this study the start setting is rather synthetic, while in reality more diverse conditions can be found.

The **constraint model** defines a mandatory temperature band for each thermal device, limiting the measured temperature to lie between a constant minimum and maximum:

$$\forall t \in \mathbb{T} : 2^\circ C = T_{min} \leq Temp.(t) \leq T_{max} = 8^\circ C$$

Additionally, for the energy storage the capacity is defined, constraining the acceptable values to lie between zero and a constant maximum:

$$\forall t \in \mathbb{T} : 0 = SOC_{min} \leq SOC(t) \leq SOC_{max} = 5 \text{ kWh}$$

Other than that, no other constraints are defined to narrow the operational limits of the system under development. In particular, the local net remains unconstrained.

The **objective model** translates the customer's autonomy goal into a cost function over the local net balances, which is to be minimized within the operational limits:

$$Cost(t) = \sum_{t'=0}^t Balance(t')^2$$

It should be noted that other formalizations are possible (e.g. linear cost functions). However, the choice of formalization mainly depends on the customer's opinion.

The **software model** is decomposed into a storage controller and individual thermal controller components. The *storage controller* defines the charge decision as a free variable, over which the system trace can be optimized.

$$Charge(t) \in \{Charge, Idle, Discharge\}$$

Consequently, the energy storage's state of charge and load on the local net can be used flexibly to counterbalance over- and underproduction.

Instead, the *thermal controllers* switch between activating and deactivating the cooling compressor in case the upper and lower temperature bounds T_{max} and T_{min} are reached.

$$Cool(t) = \begin{cases} On & \text{if } Temp(T-1) + \Delta T_+ > T_{max} \\ Off & \text{if } Temp(T-1) - \Delta T_- > T_{min} \\ Cool(t-1) & \text{else} \end{cases}$$

Consequently, all thermal devices are switching between periods of cooling and non-cooling. In particular, since all thermal devices start with the same temperature, their periods coincide.

Finally, the **equivalence model** is defined with the energy storage level as the single component of the one-dimensional equivalence vector. The rationale was that at any particular point in time it remains unclear whether to prefer more or less stored energy. If more energy is stored, future demand can be served even if the solar panel production decays. If less energy is stored, future overproduction can be reduced instead.

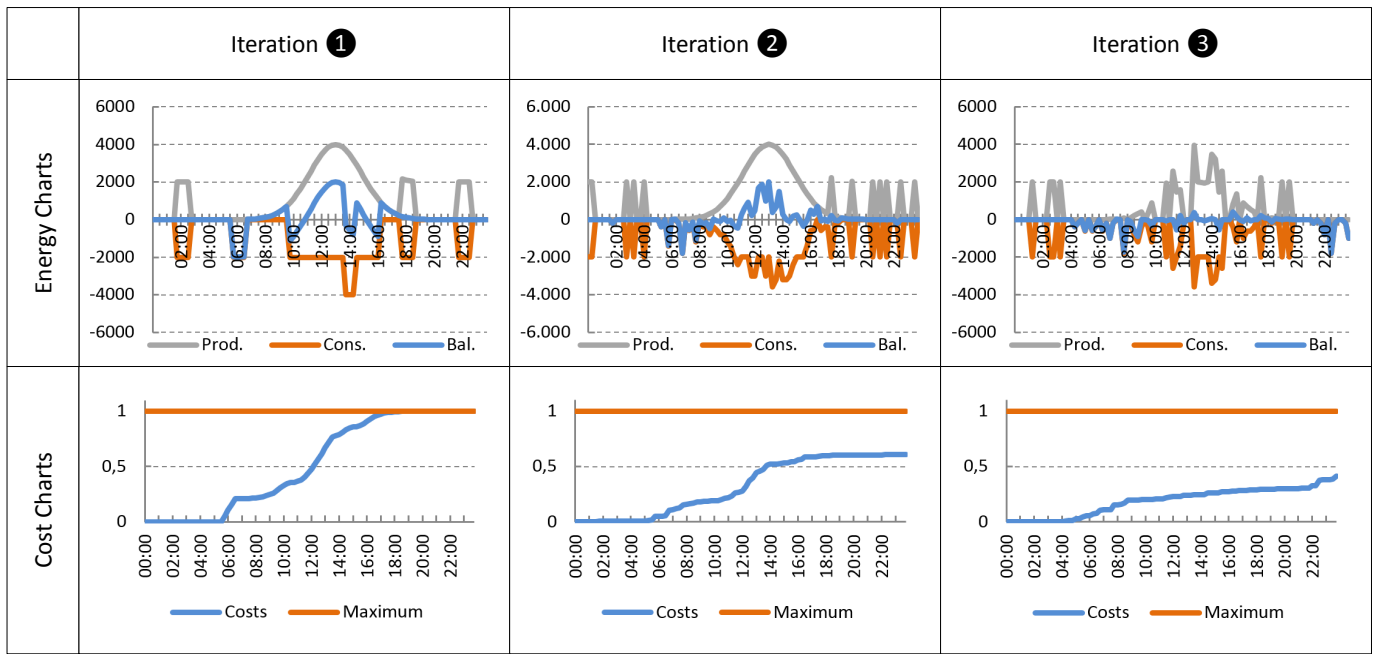


Fig. 3. Estimated system behavior over all three iterations illustrated in terms of energy production, consumption and balance as well as cost charts over time.

2) *Model Analysis:* The estimated system behavior for one day is depicted in Figure 3. The first iteration showed that in the given scenario smart energy storage control can improve the local net autonomy for three reasons: (1) Around 2:00 am the consumption of thermal devices can be counterbalanced once. However, around 6:00 am no stored energy is left to support the thermal devices a second time. (2) During day time the energy storage can be used to draw off solar panel overproduction increasing the state of charge. (3) Around 6:00 pm and 10:00 pm the stored energy can be used to counterbalance thermal device consumption again. Consequently, the cost function does not increase anymore.

B. Iteration 2

After the first iteration the customer decides that the estimated autonomy is not sufficient yet. Possible reasons could be the financial investment that needs to be made in extra line or transformer capacities. Therefore, the customer requests to add smartness to the thermal devices so that they can be turned on and off flexibly. Consequently, one non-deterministic thermal device coordinator and ten probabilistic thermal device controllers are added to the software model as depicted in Figure 2. The coordinator suggests an overall cooling probability, while the controllers take the actual cooling decisions. Also, the equivalence model is updated to use the average thermal device temperature as second component of the equivalence vector. The model details are omitted here for brevity.

The estimated system behavior is shown in Figure 3. The modification is able to improve the autonomy cost to about 60% of the previous iteration for two reasons: (1) The thermal device consumption around 6:00 am can be spread to obtain lower peak loads reducing the effect of underproduction in the morning. (2) The thermal device consumption around noon can be adjusted to solar panel production reducing overproduction during day time.

C. Iteration 3

Finally, in a third iteration the customer requests to evaluate the impact of an additional smart controller for the solar panel to improve further the autonomy of the local net. Technically, solar panels allow one to throttle production using the maximum power point tracking system [15] within the power inverter. Consequently, a non-deterministic solar controller prescribing production efficiency is added to the software model as shown in Figure 2. Again model details are omitted for brevity.

The modification allows an additional reduction of the autonomy cost to about 40% of the first iteration for one reason: During day time the solar panel overproduction can be reduced such that the system operates almost at perfect balance. Obviously, the modification does not impact the underproduction around 6:00 am. Further, the reduced overproduction comes at the cost of reduced stored energy such that around 10:00 pm the thermal device demand cannot be served anymore.

At this point the study ended with the customer deciding that the third stage was sufficient as no more expansion options were available and it delivered sufficiently better results compared with the previous two iterations.

V. DISCUSSION

In the following the presented approach is discussed with respect to modeling effort and required level of granularity.

The software model probably provides the greatest potential for saving modeling effort. In the last iteration our study contained three software component types each only described in terms of a set of possible outputs per time step omitting the actual decision logic behind. Consequently, different stages of expansion can be evaluated quickly as

soon as the context model is available. However, it should be noted that due to state space explosion more complex system models require more elaborate software models to be manageable during optimization. We believe that incremental techniques might help starting with small non-deterministic software models and gradually increasing system complexity while adjusting the degree of determinism.

Then, the equivalence models have shown to be valuable for complementing the software models through user-defined information about state clustering. In the case, their formulation took little effort comparable to the constraint and objective models. However, we are missing an appropriate method for deriving suitable equivalence models. Therefore, we try to understand their meaning through a series of different cases.

In our case we selected a time- and value-discrete causal model of the context with a rather coarse temporal resolution. Consequently, modeling effort and computational complexity are reduced allowing quick iteration. In case more detailed models such as differential equation systems are needed, the modeling effort increases. However, existing tools such as MATLAB-Simulink¹ or OpenModelica² could be integrated providing extensive component libraries. It remains unclear, though, how to cope with the computational complexity of these models during optimization. To this end, we work on methods for deriving more appropriate software and equivalence models for guiding the optimization.

Further, we have seen that constraints typically come with specific component types (e.g. temperature bands for thermal devices or state of charge limits for energy storages). In our case, constraint modeling required minimal effort, however more complex cases might require more effort. Therefore, we believe that component libraries should be shipped with integrated, parameterized constraint models.

Finally, we have shown one particular objective model formulating local net autonomy as cost function over local net balances. In general, a wide variety of objectives is expected including aspects such as operating at optimal efficiency or minimizing wear of equipment. It will be a challenge to provide a suitable method and reuse techniques dealing with this issue. To this end, we think that software quality model based techniques [16] sound promising.

VI. CONCLUSION & OUTLOOK

In this paper we proposed a rapid prototyping workflow for smart energy systems. Compared to existing approaches, our approach reduces modeling effort through use of partial system models. To obtain useful estimates of operative system behavior a derivative-free stochastic optimization method is employed. In a case study we exemplified the applicability of the approach over three iterations between customer and software engineer. We showed how to successively refine requirements along with prototypes. Consequently, we are able to obtain quickly a solid basis for decision making. However, we evaluated only time- and state-discrete models of the system. Currently, we are working on a real world case with industrial cooperation. Additionally, we will widen the scope

to e-mobility scenarios with fast, energy-efficient, collision-free vehicle routing. Finally, we will work on methods to handle larger model complexities caused by both more diverse components and near-continuous state models.

REFERENCES

- [1] S. Massoud Amin and B. Wollenberg, "Toward a smart grid: power delivery for the 21st century," *Power and Energy Magazine, IEEE*, vol. 3, no. 5, pp. 34 – 41, sept.-oct. 2005.
- [2] H. F. Hofmann and F. Lehner, "Requirements engineering as a success factor in software projects," *IEEE software*, vol. 18, no. 4, pp. 58–66, 2001.
- [3] B. Ramesh *et al.*, "Process knowledge based rapid prototyping for requirements engineering," in *Requirements Engineering, 1993., Proceedings of IEEE International Symposium on*. IEEE, 1993, pp. 248–255.
- [4] C. K. Chua, K. F. Leong, and C. C. S. Lim, *Rapid prototyping: principles and applications*. World Scientific, 2010.
- [5] G. von Laszewski, I. Foster, J. Gawor, and P. Lane, "A java commodity grid kit," *Concurrency and Computation: Practice and Experience*, vol. 13, no. 8-9, pp. 645–662, 2001. [Online]. Available: <http://dx.doi.org/10.1002/cpe.572>
- [6] Z. Vale, H. Morais, and H. Khodr, "Intelligent multi-player smart grid management considering distributed energy resources and demand response," in *Power and Energy Society General Meeting, 2010 IEEE*, July 2010, pp. 1–7.
- [7] A. Vaccaro, M. Popov, D. Villacci, and V. Terzija, "An integrated framework for smart microgrids modeling, monitoring, control, communication, and verification," *Proceedings of the IEEE*, vol. 99, no. 1, pp. 119–132, Jan 2011.
- [8] M. Faschang, F. Kupzog, R. Mosshammer, and A. Einfalt, "Rapid control prototyping platform for networked smart grid systems," in *Industrial Electronics Society, IECON 2013 - 39th Annual Conference of the IEEE*, Nov 2013, pp. 8172–8176.
- [9] A. Elsheikh, E. Widl, and P. Palensky, "Simulating complex energy systems with modelica: A primary evaluation," in *Digital Ecosystems Technologies (DEST), 2012 6th IEEE International Conference on*, June 2012, pp. 1–6.
- [10] M. Broy and K. Stølen, *Specification and development of interactive systems: focus on streams, interfaces, and refinement*. Springer-Verlag New York Incorporated, 2001.
- [11] G. Hackenberg, M. Irlbeck, V. Koutsoumpas, and D. Bytschkow, "Applying formal software engineering techniques to smart grids," in *Proceedings of the 1st International Workshop on Software Engineering Challenges for the Smart Grid*, ser. SE4SG'12. New York, NY, USA: ACM, 2012.
- [12] G. Hackenberg and D. Bytschkow, "Towards early emergent property understanding," in *Proceedings of the 1st Extreme Modeling Workshop at MODELS 2012*, ser. XM 2012. New York, NY, USA: ACM, 2012.
- [13] A. Campetelli and M. Spichkova, "Towards system development methodologies: From software to cyber-physical domain," in *Proceedings of the 1st International Workshop on Formal Techniques for Safety-Critical Systems*, ser. Electronic Proceedings in Theoretical Computer Science, P. C. Olveczky and C. Artho, Eds., vol. 105. Kyoto, Japan: Open Publishing Association, 2012.
- [14] M. Pereira and L. Pinto, "Multi-stage stochastic optimization applied to energy planning," *Mathematical Programming*, vol. 52, no. 1-3, pp. 359–375, 1991. [Online]. Available: <http://dx.doi.org/10.1007/BF01582895>
- [15] E. Koutroulis, K. Kalaitzakis, and N. Voulgaris, "Development of a microcontroller-based, photovoltaic maximum power point tracking control system," *Power Electronics, IEEE Transactions on*, vol. 16, no. 1, pp. 46–54, Jan 2001.
- [16] F. Deissenboeck, E. Jürgens, K. Lochmann, and S. Wagner, "Software quality models: Purposes, usage scenarios and requirements," in *Software Quality, 2009. WOSQ '09. ICSE Workshop on*, May 2009, pp. 9–14.

¹<http://www.mathworks.de/products/simulink/>

²<https://www.openmodelica.org/>